

TeamFlow

Systems Engineering Assignment — Design Document

Domain Model / ERD · Architecture · Business Rules ·
Design Decisions Log · Code Repository · API Overview

Kotla Sai Prakash Rao

Part 1 — Domain Model / ERD

The diagrams below use standard UML class-diagram notation: a filled diamond means **composition** (the part has no independent existence and belongs to exactly one whole — e.g. a Comment cannot exist without the Task or RCA it belongs to); a plain line with multiplicities means an ordinary **association** (a reference that can outlive either side); a dashed line to a small box hanging off an association means an **association class** — a relationship that itself carries data (used here for `TaskRelation` and `ProjectTaskLink`, which model many-to-many relationships that need their own fields, not just a join).

Why there is no class inheritance in this model. The reference style for this kind of diagram (e.g. Person → Patient/Staff → Doctor → Surgeon) uses generalisation because a Patient *is-a* Person permanently. TeamFlow's roles don't work that way: the same `User` can be an `admin` on one project and a `viewer` on another at the same time — role is contextual per project, not a fixed trait of the person. Modelling `Admin` / `Member` / `Viewer` as subclasses of `User` would fix a user's role globally, which directly contradicts the per-project RBAC requirement. Role is therefore a plain attribute on the `ProjectMember` composition, not a subclass — a deliberate choice, not an oversight.

Panel A — Core entities: users, projects, membership

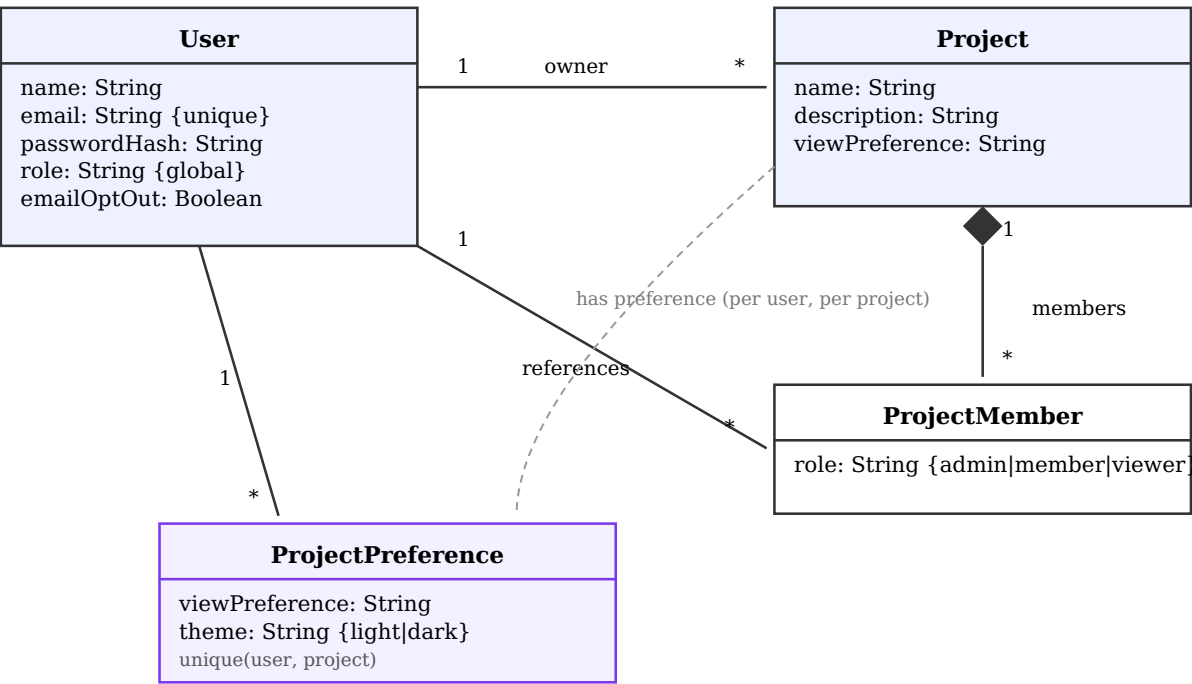


Fig. 1 — User owns Projects; a Project's `members` is a **composition** of `ProjectMember` parts (each referencing a `User`); `ProjectPreference` is an association class capturing one user's view/theme setting per project.

Panel B — Task model, dependencies, and cross-project visibility

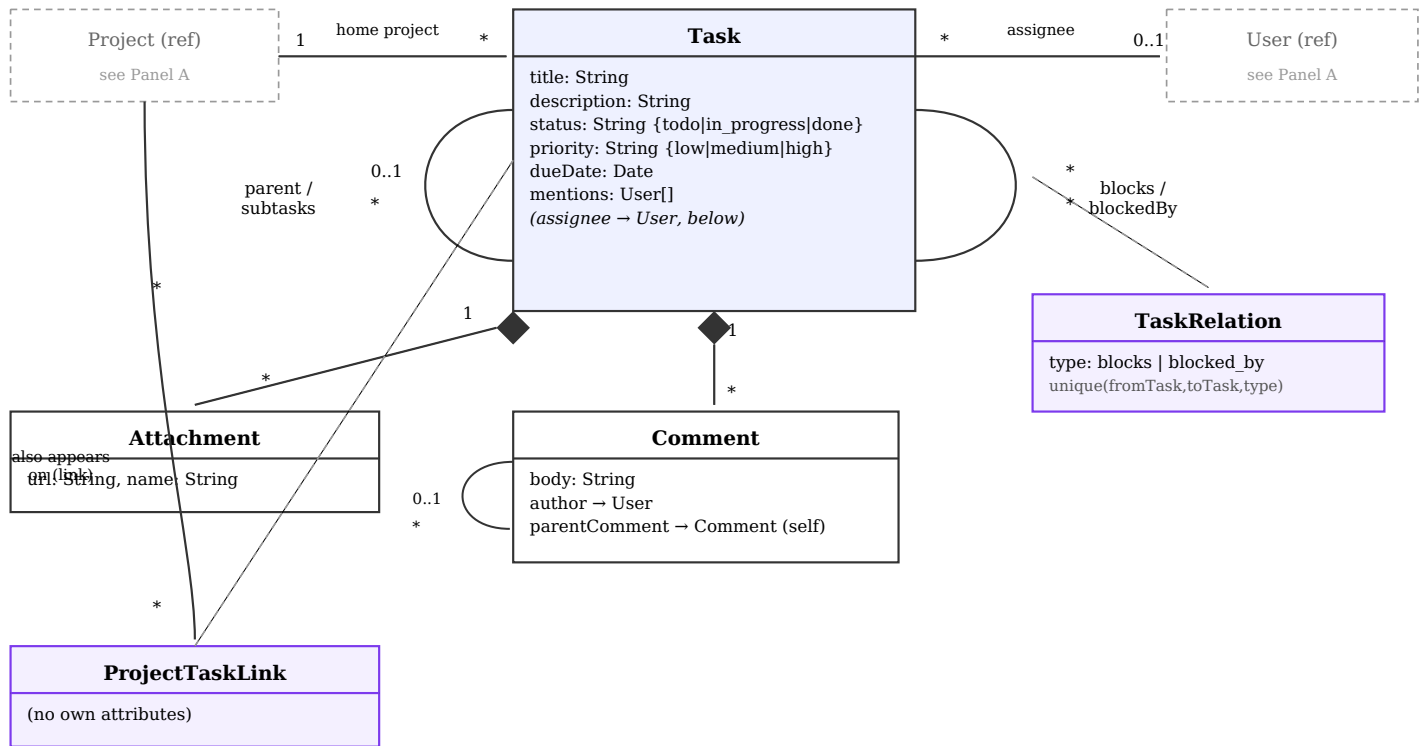


Fig. 2 — Task is composed of Comments and Attachments (they cannot exist without it); dependencies (`blockedBy` / `blocks`) and cross-project visibility are both many-to-many relationships carrying their own data, so each is modelled as an **association class** (`TaskRelation` , `ProjectTaskLink`) rather than a plain join.

Why Project-Task is a plain association, not composition. Deleting a project does not currently cascade-delete its tasks in this implementation — a task would be orphaned (still exist, referencing a deleted project) rather than removed with it. A plain association correctly reflects that actual behaviour; drawing a filled diamond here would overstate a lifecycle guarantee the code doesn't provide. This is flagged again in Known Limitations as something a production build should address (either real cascade delete or a soft-delete/archive flag).

Panel C — RCA workflow and audit trail

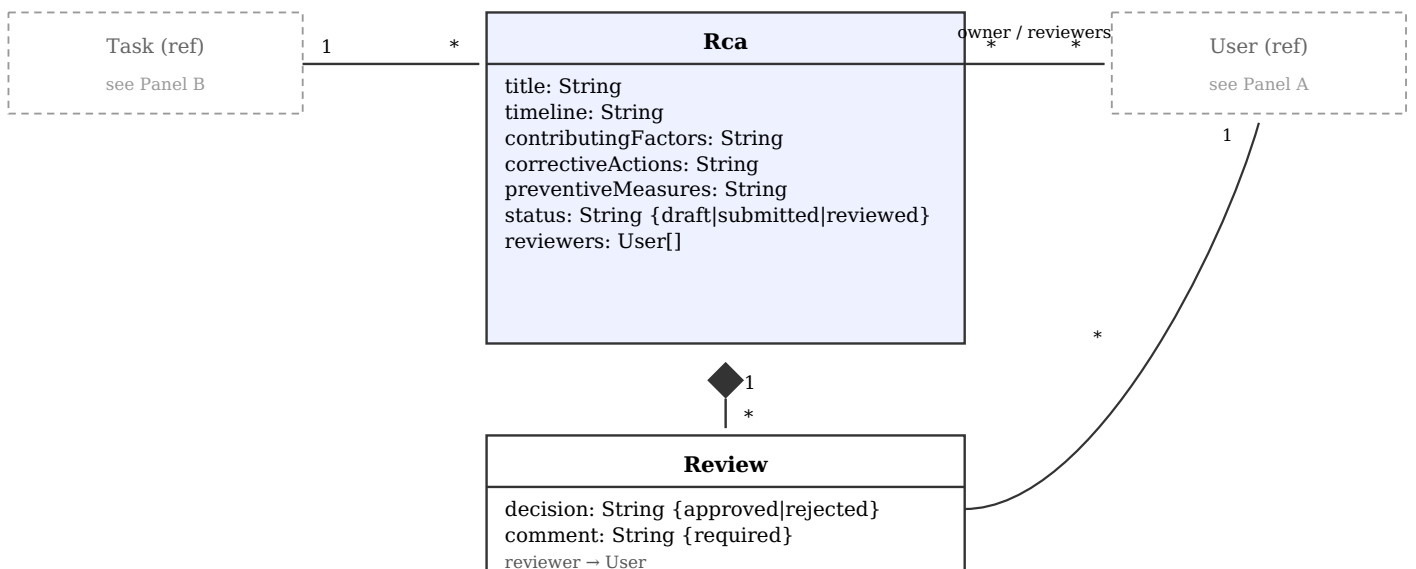


Fig. 3 — One Task can have many RCAs over time. An Rca is composed of `Review` parts (one per reviewer decision) — each Review references the reviewing `User` but cannot exist independently of its Rca.

Notification and ActivityLog (both simple, referencing `User` as recipient/actor respectively, with no composition or association-class relationships of their own) are described in the schema table below rather than diagrammed separately, to keep the panels focused on the relationships that need explaining.

Constraints

- `User.email` — unique index, enforced at the database layer.

- `ProjectPreference(user, project)` — unique compound index (Fig. 1).
- `TaskRelation(fromTask, toTask, type)` — unique compound index (Fig. 2).
- RCA `Review.comment` — required at the application layer; a decision is rejected with 400 if empty.
- Task status transition to `done` — rejected with a specific reason code if any `blockedBy` task or subtask is not `done`.

Lifecycles

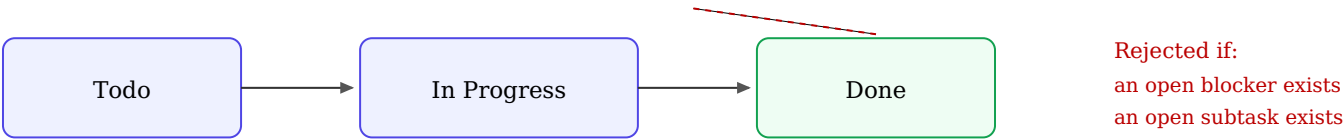


Fig. 4 — Task status lifecycle

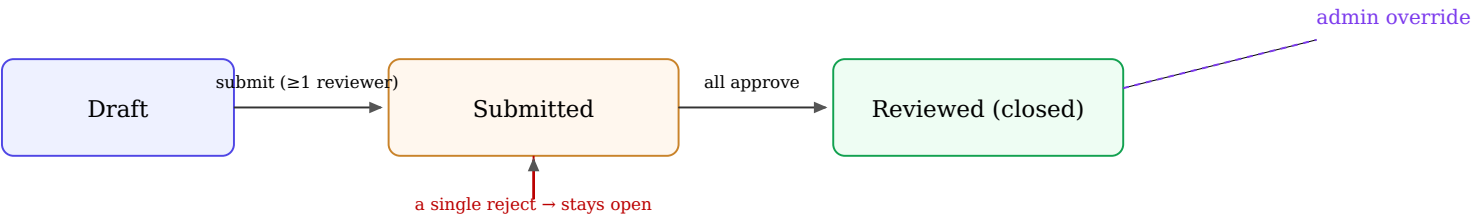


Fig. 5 — RCA status lifecycle

Part 2 — Architecture

TeamFlow is a single-page React application talking to a monolithic Node.js/Express REST API, backed by MongoDB. File attachments are stored on local disk (behind a single abstraction point) and notifications flow through an in-process event bus. Every connection below is justified against at least one alternative that was considered and rejected for this project's scale.

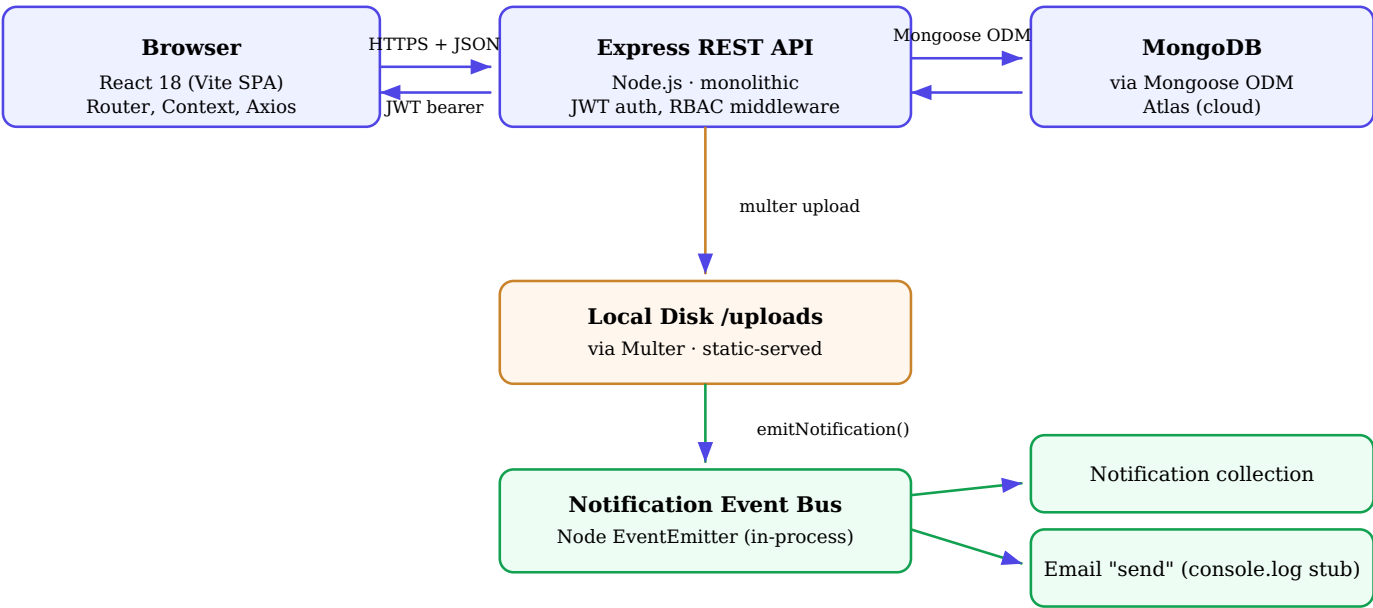


Fig. 6 — TeamFlow system architecture

Monolith vs. Microservices

Chosen: Monolith. One deployable Express app serving all resources (projects, tasks, RCA, notifications, analytics).

- **Why:** single developer/small team, single deploy pipeline, no distributed-transaction problems (e.g. "mark task done" and "write ActivityLog entry" are a single process, not a saga across services), far simpler local debugging and demoing for a coursework timeline.
- **Tradeoff accepted:** the whole app scales as one unit, and a bug in one route (as we found and fixed several times during development, e.g. a task-update route that crashed on every call) can't be isolated from the rest of the API by a service boundary.
- **Migration path:** the codebase is already organised by resource (routes/tasks.js, routes/rca.js, routes/analytics.js...), so splitting into a Task service, RCA service, and Notification service later is a matter of moving folders and adding inter-service HTTP/queue calls — not a rewrite.

MongoDB vs. SQL (PostgreSQL)

Chosen: MongoDB (via Mongoose).

- **Why:** comments, attachments, and RCA reviews are naturally nested, variable-shaped documents that are always read/written together with their parent (a task's comments are never queried independently of the task) — embedding them avoids a join for the overwhelmingly common access pattern. Schema also changed frequently during development (several fields were added mid-project, e.g. `entityId` on Notification, `parentComment` for threading) — a schemaless document store made those additive changes cheap.
- **Tradeoff accepted:** weaker join semantics. Cross-cutting queries like "all tasks across all my projects" or "which RCAs belong to project X" require either population/lookup aggregation or a denormalised reference field, rather than a SQL join. This is visible in `routes/analytics.js`, where computing project-scoped metrics requires an explicit two-step query (find accessible project IDs, then find tasks in those IDs) rather than a single joined query.

Event-Driven Notifications

Chosen: in-process event emitter, one pipeline, two listeners (bell + email).

- **Why:** decouples "a task was assigned" from "how the recipient finds out." Any route that changes state just calls `emitNotification(...)` and doesn't need to know or care whether that becomes a bell notification, an email, or (later) an SMS — new channels are a new listener, not a change to every route that can trigger one.
- **Deduplication:** keyed on `(user, type, entityId)` within a 10-minute window, so retried/duplicated triggers (e.g. a client double-submitting an update) don't spam the same recipient about the same task twice, while still allowing two different tasks assigned in quick

succession to both notify.

- **Known limitation:** Node's `EventEmitter` only fires within a single process. It is adequate for one server instance; running more than one instance behind a load balancer would require moving this to a real broker (Redis pub/sub, RabbitMQ, or SQS) so all instances see the same events.

File Storage

Chosen (for this assignment): local disk via Multer, served statically, URL stored in Mongo.

- **Why not store files in MongoDB directly:** binary blobs bloat documents and slow down every query against the collection they live in; storing a URL/reference instead keeps Task/RCA documents small and fast to read.
- **Why local disk instead of cloud storage (S3/Cloudinary) here:** zero external account/API-key setup needed to run the assignment locally or demo it; the upload route is a single, isolated module (`routes/uploads.js`), so swapping the storage backend later is a contained change, not a scattered one.
- **Tradeoff accepted — explicitly not production-ready:** files are lost on redeploy to ephemeral hosting (e.g. Heroku's ephemeral filesystem) and aren't shared across multiple server instances. Production would swap to S3/Cloudinary behind the same interface.

Part 3 — Business Rules and Functional Thinking

Task workflow

`todo` → `in_progress` → `done`. Kept intentionally small (three states) rather than adding a separate `review` state for tasks, because review-style gating already exists as its own first-class workflow for incidents (RCA) — adding a second, weaker review concept on ordinary tasks would duplicate that without adding value at this project's scope.

Dependency handling

- **Circular dependency (A blocks B, B blocks C, C blocks A):** rejected outright at the point of creating the edge. A depth-first search runs from the proposed target back to the source before the edge is persisted; if the target is reachable from the source, the request is rejected with a specific `CIRCULAR_DEPENDENCY` reason rather than silently succeeding and corrupting the graph.
- **Unresolved blocker at status-transition time:** a *hard* block — the task simply cannot move to Done. This is deliberately stricter than a "proceed anyway" warning, because letting a task report Done while a real blocker is still open would make the whole dependency graph untrustworthy for anyone relying on it.
- **Unresolved blocker at dependency-creation time:** a *soft* warning, not a block — you're allowed to record that Task A depends on Task B even if B already has its own open blockers; the system just tells you so. Blocking this would make it impossible to record a realistic dependency graph before all the upstream work is finished.

Notification behaviour

Generated on: task assignment, task status change, RCA submission, RCA review decision, and @mentions in a comment. Every trigger goes through one pipeline (`emitNotification`) so the bell and the email channel can never drift out of sync about what counts as notify-worthy. Deduplicated on (`user`, `type`, `entityId`) within a 10-minute window — narrow enough that two distinct events (e.g. two different tasks assigned close together) still both notify, but wide enough to absorb a double-submit or retry.

Review workflow (RCA)

Multiple reviewers are assigned at submit time. Each reviewer's decision (`approved` / `rejected`) requires a comment — enforced server-side, not just as a disabled button in the UI, so a review can never be a silent rubber stamp. The RCA only reaches `reviewed` (closed) once **every** assigned reviewer has approved. A single rejection is enough to keep it open, on the reasoning that an incident review is a safety-relevant document: an unresolved objection from any one reviewer should not be outvoted by the others.

Reviewer unavailable / review cannot complete

Without an escape hatch, one unresponsive reviewer could block an RCA — and by extension the incident it documents — indefinitely. TeamFlow gives a global admin two distinct override actions, both logged as a separate `ActivityLog` action (`RCA_ADMIN_OVERRIDE`) so they're never mistaken for an ordinary reviewer decision in the audit trail:

- **Reassign reviewer:** replace an unresponsive reviewer with someone else; the RCA stays in `submitted` and waits on the new reviewer's decision.
- **Force close:** close the RCA immediately regardless of outstanding reviews, for cases where waiting further has no value (e.g. the underlying task/project is being abandoned).

Both require a written `reason`, so the override itself is accountable even though it bypasses the normal sign-off gate.

Permission table

Feature	Admin (project or global)	Member	Viewer
View project boards / reports / activity	Yes	Yes	Yes
Create / edit / delete tasks	Yes	Yes	No
Add comments / attachments	Yes	Yes	Yes (read/participate, no delete of others')
Open an RCA against a task	Yes	Yes	No
Approve/reject an RCA	Only if listed as a reviewer	Only if listed as a reviewer	No
Admin override on RCA	Global admin only	No	No
Manage project members / roles	Yes (project admin/owner)	No	No
Delete project	Yes (owner)	No	No

All of the above are enforced in the Express route handlers themselves (e.g. `canEditTask` / `canViewProject` checks against the requester's project membership row, independent of anything the frontend hides) — the frontend hiding buttons is a UX convenience, not the actual security boundary.

Part 4 — Design Decisions Log

#	Decision	Alternatives considered	Tradeoff / Rationale
1	MongoDB + Mongoose	PostgreSQL + Prisma/TypeORM	Nested, variable-shaped data (comments, reviews, attachments) and a schema that changed repeatedly during development favoured a document store; accepted weaker join support in exchange for faster iteration.
2	JWT, 15-minute expiry, no refresh flow yet	Server-side sessions	Stateless auth scales horizontally with no session store; short expiry limits the blast radius of a leaked token. Tradeoff: no revocation list and no silent refresh — documented as a known limitation rather than silently ignored.
3	React Context (Auth/Theme/Toast) instead of Redux	Redux / Zustand	Three small, independent pieces of global state with no complex cross-slice logic; Redux's boilerplate (actions, reducers, store wiring) wasn't earning its keep at this scale.
4	Multer + local disk instead of Mongo GridFS or cloud storage	GridFS, S3, Cloudinary	GridFS re-introduces the "large binaries in the database" problem storing a URL was meant to avoid; cloud storage needs external account/API-key setup not justified for a graded assignment. Isolated to one route so it's a contained swap later.
5	Append-only ActivityLog collection	Overwrite fields in place with an <code>updatedAt</code> / <code>updatedBy</code> pair	An audit trail needs to answer "what changed, by whom, and to what" over time — overwriting loses history. A dedicated, append-only, generically-shaped (<code>entityType/entityId/actor/action/payload</code>) collection can log any entity without a schema change per feature.
6	Embedded comments/attachments/reviews vs. separate collections	Top-level <code>Comment</code> , <code>Attachment</code> , <code>Review</code> collections with foreign keys	These are always read/written as part of their single parent, never queried independently across parents — embedding (UML composition) avoids a join on the common path. Tradeoff: can't cheaply query "all comments by user X across every task," which the product doesn't need.
7	Both inline <code>blockedBy</code> / <code>blocks</code> arrays and a separate <code>TaskRelation</code> collection	Only the arrays; or only <code>TaskRelation</code>	The arrays give a single-document read for the cycle-detection DFS and the status-transition guard, which run on the hot path of every status update. <code>TaskRelation</code> gives an explicit, independently queryable, auditable relation record (an association class in the ERD) matching a more conventional relational design. Tradeoff: two representations of the same fact must be kept in sync (handled at write time in the same route handler).
8	In-process EventEmitter for notifications	Message broker (Redis pub/sub, RabbitMQ, SQS)	Zero infrastructure to run/demo locally; adequate for a single server instance. Explicitly flagged as needing to move to a real broker before running multiple instances, since events wouldn't be visible across processes otherwise.
9	Notification dedup key: (<code>user</code> , <code>type</code> , <code>entityId</code>)	Key on (<code>user</code> , <code>type</code>) only	Keying on type alone would incorrectly suppress a second, genuinely different notification (e.g. two different tasks assigned within the same 10-minute window) — this was an actual bug found and fixed during development, not a hypothetical.
10	Monolithic Express API	Microservices per domain (Task service, RCA service, Notification service)	Single small team, single deploy target, no need yet for independent scaling of one domain over another. Codebase is already organised by resource to make a later split mechanical rather than a rewrite.
11	Server-side RBAC enforcement (not just UI hiding)	Trust the frontend to hide buttons for restricted roles	UI hiding is a UX nicety; a determined caller can hit the API directly. Every write route re-derives the requester's project role and checks it, independent of what the frontend rendered — this was in fact missing on several routes early in development and had to be retrofitted once the gap was found.
12	Circular dependency check via DFS at write time	Allow the cycle and detect it later via a background job	A cycle makes the whole blocking graph meaningless the moment it exists (nothing could ever legitimately reach Done); rejecting at the point of creation is cheap (bounded graph size) and avoids ever persisting an invalid state.
13	Bare status-transition warning (blocker) is a hard block, not a soft "proceed anyway"	Allow forcing a task to Done past an open blocker with a confirmation dialog	Letting Done mean "probably done" instead of "actually done" would make every other feature that trusts task status (the Kanban board, reports, downstream blocked tasks) unreliable. The softer, "proceed anyway" pattern is reserved for the dependency-creation warning instead, where the graph is still being built rather than being relied upon.
14	Admin override on RCA logged as a distinct action	Log it identically to a normal reviewer decision	An audit trail that can't distinguish "a reviewer approved this" from "an admin bypassed review entirely" isn't trustworthy for its stated purpose (accountability on a safety-relevant document).
15	Project creator auto-added to <code>members</code> as admin	Rely solely on the separate <code>owner</code> field for permission checks	Found during development that permission helpers checked membership, not ownership, which would have locked the creator out of their own project's tasks. Auto-adding the owner as an admin member closes that gap without special-casing "owner" everywhere permissions are checked.
16	Role modelled as a plain attribute on <code>ProjectMember</code> , not a <code>User</code> subclass	UML generalisation: <code>Admin</code> / <code>Member</code> / <code>Viewer</code> as subclasses of <code>User</code>	A user's role is per-project, not a fixed trait — the same person is <code>admin</code> on one project and <code>viewer</code> on another simultaneously. Subclassing would fix the role globally per instance, which is incompatible with that requirement. See the Domain Model note in Part 1.
17	Project-Task modelled as a plain association, not composition	Composition (cascade-delete tasks when their project is deleted)	The current implementation does not cascade-delete a project's tasks — drawing composition in the ERD would overstate a guarantee the code doesn't provide. Flagged as a production gap in Known Limitations rather than silently modelled away.

Part 5 — Code Repository

The complete source (client + server) is in the accompanying Git repository, organised by resource on the server (`routes/` , `models/`) and by feature area on the client (`project/` , `tasks/` , `rca/` , `notifications/`) — see `README.md` for the full folder structure, setup instructions, environment variables, and known limitations, and `docs/DatabaseSchema.md` for the schema-of-record (Mongoose has no migration files, so this document plays that role).

Handling new scenarios

Scenario	How the current design would need to change
Offline-first use	Not implemented. Would require a local write-ahead queue in the client (e.g. IndexedDB) and a conflict-resolution strategy on sync — the current API assumes a live connection and returns immediate authoritative state (e.g. the dependency/blocker checks run server-side against current data, which an offline client can't validate against).
Compliance (e.g. audit retention, data residency)	<code>ActivityLog</code> 's append-only design already gives a starting point for audit retention; would need a retention policy (never delete vs. archive-after-N-days) and, for data residency, region- pinned MongoDB Atlas clusters plus a decision on whether the monolith needs to be deployed per-region too.
High scale	The two load-bearing simplifications — the in-process notification <code>EventEmitter</code> and local-disk file storage — are exactly the two things that stop working across multiple server instances. Both are already isolated behind a single module each, so the migration path (real message broker; <code>S3/Cloudinary</code>) is a contained swap, not a rewrite.
Low cost	The current design is already the low-cost option: no message broker, no object storage, no third-party email provider — everything runs on one Node process plus a MongoDB Atlas free/shared tier.
Multi-region deployment	Would need the file-storage and notification swaps above, plus a decision on whether MongoDB runs as a single global cluster (simpler, latency cost for far regions) or region-sharded (faster, more operational complexity) — not something the current single-region design addresses.

Appendix — API Overview

Method & Path	Purpose	Notes
POST /api/auth/register	Create account	Global role always defaults to <code>member</code> ; cannot be set by the client
POST /api/auth/login	Authenticate	Returns a 15-minute JWT + user profile
GET /api/me	Restore session	Used on app load to re-hydrate the logged-in user from the token
GET /api/projects	List accessible projects	Owner or member
POST /api/projects	Create project	Creator is auto-added as an admin member
GET /api/projects/:id	Project detail	Populates owner + members
PUT /api/projects/:id/members	Update membership/roles	Admin/owner only
DELETE /api/projects/:id	Delete project	Owner only
GET /api/tasks? project=&status=&assignee=	List/filter tasks	Requires view access to the project
GET /api/tasks/:id	Task detail	Populated: assignee, project, blockedBy, blocks, comments.author
POST /api/tasks	Create task	Returns <code>{ task, warning }</code> — warning is a non-blocking assignee-overload flag
PUT /api/tasks/:id	Update task	Status → done is rejected with <code>reason: OPEN_BLOCKERS OPEN_SUBTASKS</code> if applicable
DELETE /api/tasks/:id	Delete task	
POST /api/tasks/:id/dependencies	Add blockedBy/blocks edge	Rejects with <code>reason: CIRCULAR_DEPENDENCY</code> ; returns a warning if unresolved blockers exist
GET /api/tasks/:id/subtasks	List child tasks	
GET /api/tasks/:id/relations	List TaskRelation rows	
POST /api/tasks/:id/comments	Add comment (threaded, @mentions)	mentions[] triggers notifications
GET /api/project-task-links?task=	List a task's cross-project links	
POST /api/project-task-links	Link a task to another project	
GET /api/rca?project=	List RCAs (scoped to accessible projects)	
GET /api/rca/:id	RCA detail	
POST /api/rca	Create RCA against a task	Requires edit access on the task's project
PUT /api/rca/:id	Edit draft, submit, or submit a review decision	Body shape varies: <code>{submit:true, reviewers}</code> or <code>{reviewDecision:{decision,comment}}</code>
POST /api/rca/:id/override	Admin reassign/force-close	Requires <code>reason</code> ; global admin only
POST /api/rca/:id/comments	Add comment	
GET /api/notifications	List current user's notifications	
PUT /api/notifications/:id/read	Mark one read	
GET /api/analytics/summary?project=	Dashboard metrics	Completion rate, workload, velocity, project health
GET /api/analytics/export? project=&status=&assignee=	CSV export	Scoped to accessible projects + active filter
GET /api/activity?project=	Activity feed	Also supports <code>entityType</code> + <code>entityId</code> for a single task/RCA
GET /api/users?search=	User search	Backs assignee/mention/reviewer pickers
POST /api/uploads	Upload a file	<code>multipart/form-data</code> , returns <code>{url, filename}</code>
GET /api/preferences/:projectId	Get view/theme preference	Per user, per project
PUT /api/preferences/:projectId	Set view/theme preference	Upsert